

HackDNA – Hack the Login

Challenge Overview

This challenge focuses on authentication weaknesses within web applications. Login systems are one of the most targeted attack surfaces because they protect sensitive accounts and administrative functionality.

Improper authentication mechanisms can allow attackers to bypass access controls, brute-force credentials, enumerate users, or gain unauthorized access.

Objective

Identify weaknesses in the application's authentication mechanism and gain unauthorized access.

Understanding Authentication Attacks

Common authentication weaknesses include:

- Weak passwords
- Default credentials
- SQL Injection
- Username enumeration
- Missing rate limiting
- Insecure session handling
- Client-side authentication logic
- Poor password storage

Attackers typically target login systems during initial access attempts.

Reconnaissance

Begin by analyzing the login form.

Inspect:

- Request methods
- Parameters
- Cookies
- Error messages

- Hidden fields
- Authentication responses

Use browser developer tools:

F12 → Network

Observe login requests.

Example:

```
POST /login HTTP/1.1  
  
username=admin&password=admin
```

Testing Default Credentials

Applications sometimes ship with weak or default credentials.

Examples:

```
admin:admin  
admin:password  
guest:guest  
test:test
```

Failure to change default accounts is a common security issue.

Username Enumeration

Applications may reveal whether usernames exist.

Example responses:

```
Invalid password
```

versus:

```
User does not exist
```

This allows attackers to identify valid accounts before attempting password attacks.

SQL Injection Authentication Bypass

Improper input validation can allow SQL Injection.

Example vulnerable query:

```
SELECT * FROM users
WHERE username='$username'
AND password='$password';
```

Malicious input:

```
' OR '1'='1
```

Example payload:

```
username: admin
password: ' OR '1'='1
```

Resulting query:

```
SELECT * FROM users
WHERE username='admin'
AND password='' OR '1'='1';
```

The condition evaluates to true and may bypass authentication.

Brute Force Attacks

If no rate limiting exists, attackers may automate password guessing.

Example using Hydra:

```
hydra -l admin -P rockyou.txt TARGET_IP http-post-form "/  
login:username=^USER^&password=^PASS^:Invalid"
```

Brute-force attacks become practical when:

- Passwords are weak
- MFA is absent
- Lockout protections are missing

Client-Side Authentication Weaknesses

Some applications incorrectly enforce authentication logic inside JavaScript.

Example:

```
if(password == "admin123"){  
    loginSuccess();  
}
```

Attackers inspecting source code can recover credentials directly.

Session Analysis

Inspect cookies after authentication.

Example:

```
Set-Cookie: session=abc123
```

Look for:

- Predictable session IDs
- Missing HttpOnly flag
- Missing Secure flag
- Session fixation issues

Exploitation Workflow

Typical attack chain:

1. Enumerate usernames
 2. Test weak credentials
 3. Attempt SQL Injection
 4. Analyze source code
 5. Brute-force passwords
 6. Manipulate sessions
 7. Gain unauthorized access
-

Root Cause

Authentication vulnerabilities commonly arise from:

- Weak security design
 - Poor credential management
 - Insecure coding practices
 - Missing input sanitization
 - Lack of defense-in-depth
 - Improper session management
-

Security Impact

Authentication flaws may lead to:

- Account takeover
- Administrative access
- Data breaches
- Privilege escalation
- Lateral movement
- Full system compromise

Authentication weaknesses are among the highest-risk web vulnerabilities.

Detection Techniques

Manual Testing

- Analyze login responses
- Test weak credentials
- Inspect cookies
- Review source code
- Observe authentication flow

Automated Testing

Using Burp Suite:

```
Intruder → Payload Positions → Password List
```

Using Hydra:

```
hydra -L users.txt -P passwords.txt TARGET_IP
```

Using SQLMap:

```
sqlmap -u "http://TARGET/login" --forms
```

Defensive Measures

Strong Password Policies

Require:

- Long passwords
- Password complexity
- Breach-password detection
- Secure password rotation

Multi-Factor Authentication

Add additional verification layers.

Rate Limiting

Restrict repeated login attempts.

Secure Session Management

Use secure cookie settings:

```
HttpOnly  
Secure  
SameSite=Strict
```

Input Validation

Sanitize user input to prevent injection attacks.

Account Lockout Policies

Temporarily block repeated failed attempts.

Security Monitoring

Monitor for:

- Brute-force attacks
- Credential stuffing
- Suspicious login behavior
- Geographic anomalies

Real-World Relevance

Authentication weaknesses are frequently exploited in:

- Banking systems
- SaaS applications
- Corporate portals
- Administrative dashboards
- APIs

Common attack techniques include:

- Password spraying

- Credential stuffing
 - MFA fatigue attacks
 - Session hijacking
-

Key Takeaway

Authentication systems should never rely on weak validation or insecure trust assumptions.

Layered defenses are essential for protecting accounts and sensitive functionality.

Vulnerability Classification

- CWE-287: Improper Authentication
 - CWE-307: Improper Restriction of Excessive Authentication Attempts
 - CWE-89: SQL Injection
 - OWASP A01:2021 – Broken Access Control
 - OWASP A07:2021 – Identification and Authentication Failures
-

Conclusion

Authentication vulnerabilities remain one of the most dangerous weaknesses in modern applications. Poor login security can rapidly lead to unauthorized access and complete compromise.

Secure authentication requires strong validation, secure session handling, layered protections, and continuous monitoring.